

Investigating Memory Architectures for Deep Reinforcement Learning in Quadrupedal Maze Navigation

Untersuchung von Speicherarchitekturen für tiefes verstärkendes Lernen zur Navigation vierbeiniger Roboter in Labyrinthen

Master thesis in the field of study "Computational Engineering" by Leon Magnus

Date of submission: August 3, 2026

1. Review: Prof. Jan Peters, Ph.D
 2. Review: Oleg Arenz, Ph.D.
 3. Review: Nico Bohlinger, M.Sc.
- Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Leon Magnus, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB der TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§ 38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 3. August 2026

L. Magnus

Abstract

Legged locomotion has made significant progress in recent years, allowing learned policies to traverse complex and rough terrain. However, these policies typically follow velocity commands provided by a human operator or a high-level planner, leaving the robot short of full autonomy.

To navigate an unknown environment autonomously, a policy must remember information that leaves its field of view and only becomes relevant much later. Reinforcement learning commonly addresses this with memory architectures such as Long Short-Term Memorys (LSTMs) trained end-to-end with the policy; yet such unstructured memories often fail to preserve information over long horizons.

In this work, we investigate how to enhance long-term memorization for learned autonomous navigation and propose the Local Spatial Matrix Memory (LSMM), a spatially structured recurrent memory based on Hadamard memory that updates only a local window around the robot’s estimated position at each step, so that information about a place persists until the robot returns to it. To evaluate memorization over long horizons, we formulate goal-free maze exploration as an end-to-end quadrupedal locomotion problem and compare LSMM against LSTM, Transformer, and fixed-window baselines. All architectures reach comparable maze coverage, but LSMM spreads its visits most evenly and is the only architecture that remembers the maze layout, reducing the reconstruction error threefold. Finally, we deploy the learned policy on a Unitree Go2 quadruped, demonstrating fully autonomous exploration of a real outdoor maze without any map, planner, or external commands.

Zusammenfassung

Fortbewegung von Robotern hat in den letzten Jahren erhebliche Fortschritte gemacht, sodass gelernte Regelungsstrategien komplexes und unwegsames Gelände überwinden können. Diese Strategien folgen jedoch typischerweise Geschwindigkeitsbefehlen, die von einer Person oder einem übergeordneten Planer vorgegeben werden, wodurch Roboter nicht vollständig autonom agieren.

Um autonom in einer unbekannten Umgebung zu navigieren, muss eine Strategie Informationen speichern, die ihr Sichtfeld verlassen und erst deutlich später relevant werden. Im Bereich des Reinforcement Learning wird dies üblicherweise mit Speicherarchitekturen wie LSTMs adressiert, die Ende-zu-Ende mit der Strategie trainiert werden. Solche unstrukturierten Speicher können Informationen jedoch oft nicht über lange Zeithorizonte erhalten.

In dieser Arbeit untersuchen wir, wie sich die langfristige Speicherung von Informationen für gelernte autonome Navigation verbessern lässt, und stellen das LSMM vor, einen räumlich strukturierten rekurrenten Speicher auf Basis des Hadamard-Speichers, der in jedem Zeitschritt nur ein lokales Fenster um die geschätzte Position des Roboters aktualisiert, sodass Informationen über einen Ort erhalten bleiben, bis der Roboter dorthin zurückkehrt. Um die Speicherfähigkeit über lange Zeithorizonte zu evaluieren, formulieren wir die ziellose Erkundung von Labyrinthen als ein Ende-zu-Ende Fortbewegungsproblem mit einem vierbeinigen Roboter und vergleichen das LSMM mit LSTM-, Transformer- und Fixed-Window-Vergleichsarchitekturen. Alle Architekturen erreichen eine vergleichbare Erkundung des Labyrinths, jedoch verteilt das LSMM die verbrachte Zeit in jeder Labyrinthzelle am gleichmäßigsten und ist die einzige Architektur, die sich die Struktur des Labyrinths merkt und dabei den Rekonstruktionsfehler um das Dreifache reduziert. Abschließend evaluieren wir die gelernte Strategie auf einem Unitree-Go2 Roboter und demonstrieren die vollständig autonome Erkundung eines realen Labyrinths im Außenbereich ohne Karte, Planer oder externe Befehle.

Contents

1. Introduction	2
2. Foundations	5
2.1. Reinforcement Learning	5
2.2. Hadamard Memory Framework (HMF)	9
3. Related Work	11
3.1. Memory Architectures for Deep Reinforcement Learning	11
3.2. Quadrupedal Locomotion and Navigation	12
4. Methodology	14
4.1. Problem Formulation	14
4.2. Policy Architecture	15
4.3. Local Spatial Matrix Memory	16
4.4. RL Training Setup in Simulation	17
4.5. Rewards	18
4.6. Curricula	18
4.7. Domain Randomization	20
4.8. Maze Generation	20
5. Experimental Setup	22
5.1. Baselines	22
5.2. Simulation	23
5.3. Real World	26
6. Results	28
6.1. Simulation	28
6.2. Real World	32

7. Discussion	34
8. Conclusion & Outlook	35
8.1. Conclusion	35
8.2. Outlook	35
A. Appendix	42

Figures and Tables

List of Figures

1.1. Real-world exploration of the Unitree Go2 and its corresponding matrix memory.	3
4.1. Actor architecture overview with the proposed LSMM memory module. . .	15
5.1. Top-down view of the maze layout chosen for visual policy analysis. . . .	25
5.2. Top-down views of the maze layouts used in real-world deployment. . . .	26
6.1. Average maze coverage for each policy at the end of an episode.	28
6.2. Average maze cell visit distribution for each policy at the end of an episode.	29
6.3. Maze reconstruction error for each policy over the course of an episode. . .	30
6.4. Predicted maze layout at three times for each policy during an evaluation episode.	31
6.5. Auxiliary prediction of spatially varying environment properties from . . .	32
6.6. Time spent in each cell by our deployed LSMM policy.	33
6.7. Maze reconstruction error over deployment time of our LSMM policy. . . .	33
A.1. Curriculum convergence during training for all approaches	46
A.2. Average reward during training for all approaches	46
A.3. Average bias reward during training for all approaches	47

A.4. Average cell visit reward during training for all approaches	47
A.5. Average maze reconstruction reward during training for all approaches . .	48
A.6. Average episode length during training for all approaches	48
A.7. Training wall-clock time for all approaches	49

List of Tables

A.1. Hyperparameters used in all experiments.	43
A.2. Reward terms and coefficients used in all experiments.	44
A.3. Domain Randomization (DR) and observation noise parameters used in all experiments	45

Abbreviations, Symbols and Operators

Acronyms

Notation	Description
BCE	Binary Cross Entropy.
CNN	Convolutional Neural Network.
DR	Domain Randomization.
DRL	Deep Reinforcement Learning.
GAE	Generalized Advantage Estimation.
GRU	Gated Recurrent Unit.
HMF	Hadamard Memory Framework.
LSMM	Local Spatial Matrix Memory.
LSTM	Long Short-Term Memory.
MAE	Mean Absolute Error.
MJX	MuJoCo XLA.
MLP	Multilayer Perceptron.
NLP	Natural Language Processing.
POMDP	Partially Observable Markov Decision Process.



Notation	Description
PPO	Proximal Policy Optimization.
RL	Reinforcement Learning.
ViT	Vision Transformer.

1. Introduction

Thanks to breakthroughs in Deep Reinforcement Learning (DRL), legged robots can now traverse rough terrain, jump over obstacles, and navigate toward goal positions using policies trained in highly-parallelized simulations or directly in the real world [1, 2, 3, 4, 5]. These policies usually follow velocity commands or short-horizon waypoints [6, 7], leaving the higher-level planning to a human operator and making the system not fully autonomous. Autonomous exploration of an unknown environment is a fundamentally different problem. The robot must not only control and reason about its body in a local frame, but also decide where to go and remember where it has already been in order to navigate and explore efficiently [8]. These additional requirements turn locomotion into a long-horizon, partially observable navigation problem in which the current local observation is not enough [9].

Memory-augmented policies address this problem by leveraging an observation-action history with architectures such as LSTMs [10], Gated Recurrent Units (GRUs) [11], Transformers [12], or simply concatenating the history for an Multilayer Perceptron (MLP) [13]. However, unstructured recurrent states, as in LSTMs or GRUs, easily mix or overwrite spatially distinct information, as credit assignment is tricky, and finite-context approaches, like Transformers or concatenation, discard old observations in less than a second with the typical history lengths [13, 14]. Classical navigation systems avoid this problem by explicitly registering observations in spatial maps, but typically separate mapping, planning, and locomotion into individual components, losing out on potential end-to-end efficiency [15, 16, 17].

In this work, we investigate the performance of memory-augmented policy architectures in quadrupedal maze navigation, in which remembering where to walk is crucial. We introduce LSMM, a novel recurrent architecture, test it against unstructured recurrent and finite-history baselines, and compare the approaches on exploration efficiency and long-term memory capability. The matrix memory leveraged by LSMM is aligned with the environment’s frame and indexed using a position estimate, leading to a semi-latent

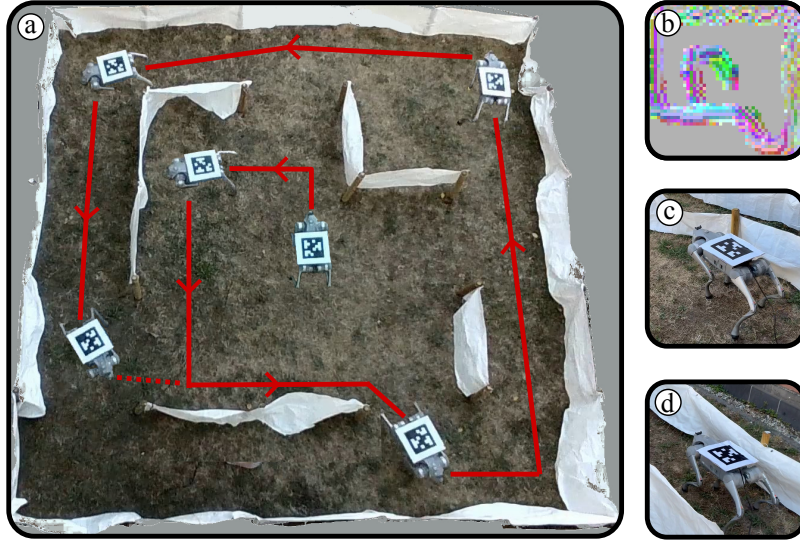


Figure 1.1.: (a) The Unitree Go2 autonomously explores a previously unseen 5×5 m outdoor maze. The red line shows the walked path. (b) The matrix memory at the end of the run. The policy has written into the cells along its path, shown with the three memory channels interpreted as RGB colors, while unwritten cells keep their initial value in gray. (c) and (d) The robot navigating through the maze.

hidden state. At every step, the policy modifies only a small window of the memory around the robot's position through learned calibration and update matrices, while the rest of the memory remains untouched. A Convolutional Neural Network (CNN) [18] encoder transforms the spatial memory into features used by both the policy and an auxiliary maze-reconstruction head. This feature extraction combines the information persistence of index-based updates and the locality of map-based navigation with learned latent representations and end-to-end locomotion control, without any explicit path planning or waypoint commands.

We formulate long-horizon, goal-free maze exploration as an end-to-end locomotion problem and train our method for the Unitree Go2 quadruped in procedurally generated mazes. Our experiments provide comprehensive empirical evaluations in simulation and a deployment in the real world, demonstrating the capabilities of LSMM in maze reconstruction, exploration efficiency, robustness to localization noise, and generalization

to unseen mazes.

Figure 1.1(a) shows our LSMM-based policy deployed in the real world. The policy starts in the middle of the maze and autonomously explores it without any provided commands. Over time, the matrix memory visualized in Figure 1.1(b) is written, and the local update property of the memory can be directly recognized from the path the agent took, as only the traversed cells have changed. Figure 1.1(c) and (d) show close-up shots of the Unitree Go2 traversing the maze while handling uneven terrain.

2. Foundations

2.1. Reinforcement Learning

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns behaviors through trial-and-error interaction with an environment. At each timestep the agent receives information about the environment, selects an action, and in return obtains a scalar reward signal together with new information about the resulting situation. The objective of the agent is not to imitate given demonstrations, but to discover a behavior that maximizes the cumulative reward it collects over time. This makes RL particularly suited for control problems such as legged locomotion, where an explicit specification of the desired low-level behavior is difficult, but the quality of a behavior can be assessed through a reward function.

2.1.1. Partially Observable Markov Decision Processes

We formulate the end-to-end locomotion problem as a Partially Observable Markov Decision Process (POMDP), defined by the tuple

$$\mathcal{P} = (\mathbf{S}, \mathbf{A}, \mathbf{\Omega}, \mathbf{P}(s' | s, a), \mathbf{O}(o' | s', a), \mathbf{R}(s, a), \gamma).$$

Here, $\mathbf{S}$ denotes the set of states, $\mathbf{A}$ the set of actions, and $\mathbf{\Omega}$ the set of observations. The transition function $\mathbf{P}(s' | s, a)$ describes the probability of reaching state $s' \in \mathbf{S}$ when taking action $a \in \mathbf{A}$ in state $s \in \mathbf{S}$. The reward function $\mathbf{R}(s, a)$ assigns a scalar reward to the agent depending on the current state s and the taken action a , while the discount factor $\gamma \in [0, 1]$ trades off immediate against future rewards.

A POMDP extends the Markov decision process (MDP) with respect to what the agent can perceive of the environment. In an MDP the agent has access to the full state s and derives

its action directly from it. In a POMDP, in contrast, the agent only receives an observation $o \in \Omega$, generated by the observation function $\mathbf{O}(o' \mid s', a)$, which maps the next state s' and the action a to the next observation o' . The observation is thus a noisy, partial estimate of the underlying state. This formulation models real-world robotic settings more closely, since onboard sensors are noisy and only capture a part of the true environment state.

The goal in this setting is to learn a policy π that maximizes the expected sum of discounted rewards:

$$\begin{aligned} G_t &= \sum_{k=0}^T \gamma^k r_{t+k}, \\ J &= \mathbb{E}_{\pi} [G_0], \end{aligned} \tag{2.1}$$

where G_t denotes the discounted return from timestep t onwards.

2.1.2. Value Functions and Bellman Equations

To reason about the quality of a policy π , two quantities are commonly introduced. The value function

$$V^{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid s_t = s] \tag{2.2}$$

describes the expected discounted return when starting in state s and following policy π afterwards. Analogously, the state-action value function

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid s_t = s, a_t = a] \tag{2.3}$$

describes the expected discounted return when taking action a in state s and following π thereafter. Both functions can be written recursively via the Bellman expectation equations, e.g.,

$$V^{\pi}(s) = \mathbb{E}_{\pi} [r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t = s]. \tag{2.4}$$

Based on Bellman's principle of optimality which states that every sub-policy of an optimal policy must itself be optimal dynamic programming methods such as policy iteration can converge from any policy π to an optimal policy π^* . However, these methods require access to the fully observable state s and the evaluation of a maximization over the action space. In a POMDP with continuous states, observations, and actions, as in our setting, such a maximization is infeasible. From this limitation policy-based methods are motivated, which instead directly optimize a parameterized policy π_{θ} with respect to the expected return.

2.1.3. Policy Gradient Methods

Policy gradient methods represent the policy as a parametric function π_θ , e.g. a neural network with parameters θ , and optimize the objective

$$\theta^* = \arg \max_{\theta} J(\theta), \quad J(\theta) = \mathbb{E}_{\pi_\theta} [G_0] = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (2.5)$$

via gradient ascent. The Policy Gradient Theorem [19] provides an expression for the gradient of this objective:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_{\theta} \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t)], \quad (2.6)$$

where $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$ denotes the advantage function. The advantage measures how much better taking action a_t in state s_t is compared to the expected behavior of the policy, and reduces the variance of the gradient estimate compared to using raw returns. In practice, actor-critic architectures are used, in which a critic network estimates V^π to compute advantage estimates, while the actor network represents the policy itself.

2.1.4. Proximal Policy Optimization

Proximal Policy Optimization (PPO) [20] is an on-policy actor-critic algorithm and the current de-facto standard for learning locomotion policies in simulation [21, 7]. Plain policy gradient updates are sensitive to the step size as large updates can drastically change the policy and destabilize training. PPO addresses this by limiting how far the new policy can deviate from the policy that collected the data. Defining the probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad (2.7)$$

PPO maximizes the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.8)$$

where $\hat{A}_t$ is an estimate of the advantage, typically computed with Generalized Advantage Estimation (GAE) [22], and ϵ is a hyperparameter controlling the clipping range. The clipping removes the incentive to move the probability ratio outside the interval $[1 - \epsilon, 1 + \epsilon]$, resulting in conservative and stable policy updates. Due to its stability, simplicity, and its suitability for massively parallel simulation, PPO is the algorithm of choice in this work.

2.1.5. Memory-Augmented Reinforcement Learning

In a POMDP, a single observation o_t is in general not sufficient to act optimally, since it only provides a noisy, partial view of the state. It can therefore be necessary to aggregate information over time—for instance, in a navigation setting, to know whether a given path has already been taken. Formally, this means the policy should not only be conditioned on the current observation but additionally on some representation $\mathbf{M}_t$ of the past,

$$\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t, \mathbf{M}_t), \quad (2.9)$$

which we refer to as a memory-conditioned policy. In the literature, two main approaches exist to construct such a memory.

Memory Through History Concatenation And Sequence Modeling

The first approach represents the memory explicitly as the history of past interactions, $h_t = (o_{t-H}, a_{t-H}, \dots, o_t)$, and feeds it into the policy. In its simplest form, a fixed window of past observations is concatenated and encoded into a feature vector from which the next action is derived. Parisotto et al. apply transformer encoders to the observation history so that the agent can learn at each step which parts of the history are currently relevant, and report more stable training than recurrent baselines [23]. A drawback of these approaches is that the accessible memory is bounded by the sequence length, and that computational cost and memory consumption grow rapidly with longer histories, which complicates long-horizon tasks.

Memory Through Recurrence

The second approach compresses the history into a hidden state that is updated at every timestep by a recurrent neural network. Instead of storing the raw history, the network learns to keep, update, or forget information as new observations arrive, yielding a constant per-step cost independent of the episode length. The most common implementations are LSTMs [10] and GRUs [24], both of which are widely used in robotic RL problems to encode observations over time. While the concrete update rules of these architectures differ, many recurrent memory mechanisms can be unified under a single matrix-based update of the form stated as the Hadamard Memory Framework (HMF) which is explained in the following section.

2.2. Hadamard Memory Framework (HMF)

In this section, we introduce the HMF, which later serves as the foundation for a new update rule for recurrent matrix-based memory. From now on, we assume matrix-based memory approaches, which differ from most off-the-shelf recurrent architectures such as GRU or LSTM. Instead of compressing all information into a single hidden state vector, matrix-based memories potentially offer additional structure through their extra dimensions. Moreover, architectures from image processing, such as CNNs or approaches similar to the Vision Transformer (ViT) [25], can be used to leverage this structured memory and extract important information from it. We state this information retrieval process as

$$\mathbf{z}_t = m(\mathbf{M}_t, \mathbf{x}_t), \quad (2.10)$$

in which m represents a parametric function that is learned to extract important features from the memory. m can be viewed as an input-dependent query, extracting relevant information conditioned on the current input context $\mathbf{x}_t$ at the given timestep. The resulting feature vector $\mathbf{z}_t$ then serves as the memory enhancement for DRL policies, where the memory component is learned end-to-end through gradients from the task objective.

The HMF [26] unifies a large class of existing memory writing rules under a single formulation. At each timestep, the memory is updated as

$$\mathbf{M}_t = \mathbf{M}_{t-1} \odot \mathbf{C}_t + \mathbf{U}_t, \quad \mathbf{C}_t = C_\theta(\mathbf{x}_t), \quad \mathbf{U}_t = U_\varphi(\mathbf{x}_t), \quad (2.11)$$

where $C_\theta, U_\varphi : \mathbb{R}^D \rightarrow \mathbb{R}^{H \times H}$ are parameterized functions of the current input and $\odot$ denotes the Hadamard (element-wise) product. The *calibration matrix* $\mathbf{C}_t$ determines which components of the previous memory are weakened, retained, or strengthened, and by how much they are scaled, while the *update matrix* $\mathbf{U}_t$ contains the new content to be written into the memory. Two properties make the Hadamard product particularly attractive as the core operator of this framework. First, it is computationally cheaper than a full matrix product. Second, it operates on each memory element individually and therefore prevents mixing the contents of different memory cells during calibration and update, which would occur under a matrix product. Unrolling the recurrence in Equation 2.11 yields the closed-form expression

$$\mathbf{M}_t = \mathbf{M}_0 \odot \prod_{i=1}^t \mathbf{C}_i + \sum_{i=1}^t \mathbf{U}_i \odot \prod_{j=i+1}^t \mathbf{C}_j, \quad (2.12)$$

where the products are taken element-wise. This form makes the role of the calibration explicit: the contribution of the information written at timestep i to the current memory is modulated by the cumulative calibration $\prod_{j=i+1}^t \mathbf{C}_j$. Setting the corresponding entries close to zero erases a no-longer-relevant experience (forgetting), whereas entries around or above one preserve or reinforce it (strengthening). In the degenerate case $\mathbf{C}_t = \mathbf{1}$ for all t , the memory neither forgets nor strengthens and merely accumulates new content over time. Since retrieving a specific past event from such an ever-growing sum requires the query to be approximately orthogonal to the keys of all other stored events, accumulation without calibration becomes increasingly ill-posed once the episode length greatly exceeds the memory dimension. An input-dependent calibration is therefore essential for selectively managing memory over long horizons [26].

Several existing memory models arise as special cases of the HMF. For instance, the matrix-based LSTM variant (mLSTM) updates its memory as

$$\mathbf{M}_t = f(\mathbf{x}_t) \mathbf{M}_{t-1} + i(\mathbf{x}_t) \mathbf{v}(\mathbf{x}_t) \otimes \mathbf{k}(\mathbf{x}_t), \quad (2.13)$$

where f and i are scalar forget and input gates, and $\mathbf{v}$ and $\mathbf{k}$ map the input to value and key representations combined via the outer product $\otimes$. This corresponds to the choice $\mathbf{C}_t = f(\mathbf{x}_t) \mathbf{1}$ and $\mathbf{U}_t = i(\mathbf{x}_t) \mathbf{v}(\mathbf{x}_t) \otimes \mathbf{k}(\mathbf{x}_t)$ in Equation 2.11. Similarly, fast-weight memories and linear Transformers can be reformulated within the framework through appropriate choices of $\mathbf{C}_t$ and $\mathbf{U}_t$ [26].

The flexibility of the calibration, however, comes at the cost of a training challenge. The gradients of the parameters with respect to early timesteps are governed by the cumulative products $\prod_j \mathbf{C}_j$ in Equation 2.12. If the calibration matrix is fixed or independent of the input, these products either diverge (entries with magnitude greater than one) or vanish (entries with magnitude smaller than one) as the horizon grows, resulting in numerical instability or the loss of learning signal from early, potentially crucial, observations [26]. Consequently, the calibration should be input-dependent while its entries oscillate around a value of one, such that the expected cumulative product remains bounded, i.e., $\mathbb{E}[\prod_{t=1}^T \mathbf{C}_t] \approx \mathbf{1}$ as $T \rightarrow \infty$. The Stable Hadamard Memory proposed by [26] instantiates this principle with a calibration design whose entries are bounded to $[0, 2]$ and oscillate around one, thereby mitigating vanishing and exploding gradients while still allowing context-dependent forgetting and strengthening.

3. Related Work

In this chapter, we review related work from two fields relevant to our problem. First, we discuss memory architectures for DRL. Second, we survey work on quadrupedal locomotion and navigation. Finally we position our problem and the proposed LSMM architecture at the intersection of both fields.

3.1. Memory Architectures for Deep Reinforcement Learning

Partial observability in DRL is commonly addressed by conditioning the policy on a finite observation history or a recurrent latent state [27]. Parisotto et al. bring transformer architectures from Natural Language Processing (NLP) into DRL and show that the canonical transformer is difficult to optimize with RL objectives, which they address by reordering the layer normalization and introducing gating layers to stabilize training and surpass LSTM agents on memory-based tasks [23]. A fundamentally different use of such sequence models is to frame RL itself as a sequence modeling problem. Chen et al. introduce the Decision Transformer, which is trained on trajectories of returns, states, and actions and, conditioned on a desired return, predicts the actions that achieve it, which replaces traditional RL optimization entirely [28]. As the quadratic cost of attention becomes prohibitive for long trajectories, follow-up work replaces or combines the transformer backbone with the Mamba state space model, which processes sequences with linear complexity [29, 30]. However, these approaches operate in an offline RL setting and require a dataset of pre-collected trajectories, which conflicts with our task, where useful data only arises through exploration itself. As the observation history is processed as a sequence, they are furthermore subject to the same context length limitation as the other attention-based approaches. Le et al. show that LSTMs, GRUs, and many other recurrent architectures update their memory in the same underlying way, calibrating the previous memory element-wise and then writing new information additively [26].

They formalize this update rule as the HMF which was introduced in section 2.2 and derive Stable Hadamard Memory, whose calibration keeps the memory from decaying or exploding over long horizons. A recent instance of this family is the xLSTM of Beck et al., which extends the LSTM with exponential gating and a matrix memory whose covariance update follows the same scheme of calibrating the previous state and additively writing new information, and as a recurrent architecture with a constant-size state it remains directly applicable to online RL [31]. However, all of these memories aggregate observations over time and maintain no explicit link between the stored information and the spatial location where it was acquired.

Spatial neural memories address this limitation by aligning the latent state with the geometry of the environment. Neural Map stores features in a two-dimensional memory indexed by the agent’s position for maze navigation [29]. MapNet similarly constructs a self-centered latent map and learns to register new observations within it [30].

3.2. Quadrupedal Locomotion and Navigation

Quadrupedal locomotion refers to the challenge of traversing rough, irregular, and unknown terrain with four-legged robotic systems, which requires the robot to sense its environment and adapt to highly dynamic situations with potentially unpredictable disturbances. In recent years, quadrupedal locomotion has made significant progress by leveraging DRL, as heavily parallelized simulation frameworks such as MuJoCo XLA (MJX) [32] and Isaac Sim [33] have made data collection significantly faster and enable the learning of locomotion policies within minutes [21]. These methods allow agents to acquire robust and agile policies in highly complex environments [21, 34], where most approaches solve the traversal by following a provided velocity command in the robot’s x, y, and yaw direction.

More recent works replace the velocity command with waypoints, given either as coordinates for the robot’s base [35] or as foothold positions [36], which forces the agent to master highly complex terrain involving jumping, climbing, and balancing, and shows that giving the agent more navigational freedom to reach a position leads to more agile and complex skills. However, the navigation in these works remains local, as the agent is only given some freedom to move toward a prescribed waypoint rather than deciding where to go on its own. The degree of this freedom varies, ranging from rather restricted motion in the parkour works [35, 36] to more navigational freedom when locomotion and navigation are learned jointly, which leads to a less restricted solution space and

thereby to more agile and varying gaits depending on the faced terrain [7]. Locomotion and navigation are therefore still often viewed as separate problems in current research, with navigation reduced to following externally provided local targets.

Classical legged-navigation stacks make this separation explicit by putting the locomotion policy as the last layer in their pipeline, where it executes velocity or waypoint commands produced by separate localization, mapping, and planning modules [15, 16, 17]. Learning-based approaches have tried to merge parts of this pipeline, as Rudin et al. train a single policy that reaches a commanded target position within a given time [7] and Zhang et al. learn a navigation policy that steers a separate locomotion policy to such a target [37]. Active Neural SLAM goes one step further and combines a learned mapping module with global and local policies, but still keeps an explicit analytical planner between the map and the executed actions [38]. In all of these cases, the decision where to move ultimately comes from an external planner or operator, which for exploration typically builds an occupancy map and repeatedly picks the next target on the frontier between the mapped and unexplored space [8].

In this work, we remove this last remaining component and view locomotion and navigation as one joint problem, hypothesizing that this leads to better traversal and more efficient exploration. Without an external planner, however, the policy itself must take over what the occupancy map provided, namely remembering which parts of the environment it has already seen and deciding where to move next. This turns the navigation task into a memory problem and is precisely why the memory architectures discussed in the previous sections are central to this work. Our LSMM keeps the spatial indexing of the spatial neural memories but does not need the analytical planner of Active Neural SLAM, since its memory features enter the locomotion policy directly. Unlike Neural Map, which writes a single feature vector into the agent’s current cell, it retrieves a whole window of cells around the robot’s estimated position, applies the HMF update rule inside this window, and leaves the rest of the memory unchanged. Memory and locomotion policy are trained jointly, and an auxiliary head reconstructs the maze layout from the memory features, which encourages the memory to retain task-relevant information about the maze. The following chapter introduces the architecture and training setup that realize this joint formulation.

4. Methodology

We propose the LSMM, a spatially indexed latent memory that allows a single policy to remember its environment, decide where to go, and control the robot’s joints at the same time. First, we formalize the exploration task and the policy architecture. Then, we introduce the local memory update of LSMM and describe how memory, encoders, and policy are trained jointly with PPO [20] and an auxiliary maze reconstruction objective. Finally, we present the key components of our simulation environment for learning an end-to-end navigation policy that transfers zero-shot to the real world, discussing the rewards, the curricula, the domain randomization, and the maze generation in detail.

4.1. Problem Formulation

We consider autonomous exploration of unseen mazes with the Unitree Go2 quadruped. The robot starts at the center of a 10×10 m maze built from a 5×5 grid of 2×2 m cells and must visit as many cells as possible within an episode of 70 s. At deployment, the policy receives no goal position, waypoint, or velocity command. Since the current observation does not reveal which parts of the maze the robot has already visited, the task is a POMDP in which the policy must remember information over time. We therefore learn a memory-augmented policy $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t, \mathbf{M}_t)$ that maximizes the expected discounted return $\mathbb{E} [\sum_t \gamma^t r_t]$, where $\mathbf{M}_t$ is the spatial memory introduced in section 4.3 and r_t is the reward described in section 4.4.

The observation $\mathbf{o}_t$ consists of a three-dimensional command $\mathbf{c}_t$ explained in section 4.6, which is only used during training and constant at deployment, the proprioceptive state $\mathbf{o}_t^p$ comprising joint positions, joint velocities, the previous action, the angular velocity of the trunk, and the projected gravity vector, and an exteroceptive observation $\mathbf{o}_t^h$ of terrain height samples around the trunk. The height samples are distributed in a radial pattern of 25 rays linearly spaced over 360° , where along each ray of 1 m length 10 points

are sampled at equal distances, leading to a total of 250 height samples. In addition, the memory module receives a noisy estimate $\hat{\mathbf{p}}_t = (\hat{x}_t, \hat{y}_t, \hat{\psi}_t)$ of the robot's position and heading in the environment frame. The critic has access to privileged information, namely the linear velocity of the trunk, its height above ground, the contact states of the feet, the ground-truth position, the per-cell visit counts, and a wider map of height samples. The actions $\mathbf{a}_t$ are target joint position offsets to a nominal standing configuration, tracked by a PD controller at 50 Hz.

4.2. Policy Architecture

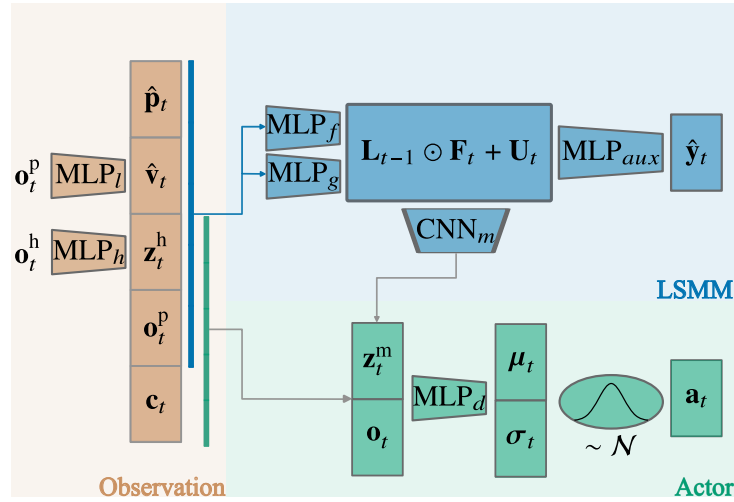


Figure 4.1.: Actor architecture with the proposed LSMM memory module. Both branches share $\mathbf{z}_t^h$ and $\mathbf{o}_t^p$, while the memory branch additionally uses noisy position and velocity estimates $\hat{\mathbf{p}}_t$ and $\hat{\mathbf{v}}_t$ to update the matrix memory $\mathbf{L}_t$. A CNN then extracts $\mathbf{z}_t^m$ from $\mathbf{L}_t$ and an auxiliary MLP predicts the maze wall layout $\hat{\mathbf{y}}_t$. The actor maps $\mathbf{z}_t^m$ and its observation $\mathbf{o}_t$ to μ_t and σ_t , from which $\mathbf{a}_t$ is sampled.

Figure 4.1 shows the policy architecture, which consists of two parts, the memory module and the actor. Since the Go2's IMU cannot measure linear velocity directly, an MLP first estimates it from the proprioceptive observation as $\hat{\mathbf{v}}_t = l(\mathbf{o}_t^p)$. A second MLP encodes the height samples into a feature vector $\mathbf{z}_t^h = h(\mathbf{o}_t^h)$. Then, the memory module takes

$[\mathbf{o}_t^p, \mathbf{z}_t^h, \hat{\mathbf{v}}_t, \hat{\mathbf{p}}_t]$ and writes them into the LSMM, as described in section 4.3. A CNN then encodes the updated memory into a feature vector $\mathbf{z}_t^m = m(\mathbf{M}_t)$. Finally, the action is sampled from a Gaussian distribution

$$\mathbf{a}_t \sim \mathcal{N}(\boldsymbol{\mu}_t, (\boldsymbol{\sigma}_t^2)), \quad (\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t) = d([\mathbf{c}_t, \mathbf{o}_t^p, \mathbf{z}_t^h, \mathbf{z}_t^m]).$$

Together, these networks form the policy π_θ , where θ collects the parameters of the velocity estimator l , the encoders h and m , the memory write functions f and g , and the actor d .

We train with an asymmetric actor-critic setup [39]. The critic is an MLP without memory. It receives the privileged observations from section 4.1, where the per-cell visit counts summarize the exploration progress that the actor has to remember. An auxiliary head predicts the wall layout of the maze from the actor features at every step. The full layout can only be recovered from information gathered earlier in the episode, so predicting it requires the memory to store what the robot has seen. At deployment, the critic and the wall prediction are dropped, and only π_θ runs on the robot.

4.3. Local Spatial Matrix Memory

The LSMM is a tensor $\mathbf{M}_t \in [0, 1]^{H \times W \times C}$ whose first two dimensions are aligned with the environment frame. We use $H \times W = 50 \times 50$ cells with $C = 3$ channels, so one cell covers 0.2×0.2 m of the maze. The channels carry learned features without a predefined meaning, and all entries are initialized to 0.5 at the beginning of an episode.

Updating the full memory at every step, as done by the recurrent architectures in the HMF [26], would constantly rewrite information about locations the robot currently cannot observe. Therefore, LSMM only updates a window $\mathcal{W}_t$ of 5×5 cells centered on the memory cell of the position estimate $\hat{\mathbf{p}}_t$, corresponding to 1×1 m around the robot. Let $\mathbf{L}_{t-1} = \mathbf{M}_{t-1}[\mathcal{W}_t]$ denote the current content of this window. Two MLPs f and g compute a calibration tensor $\mathbf{F}_t$ and an update tensor $\mathbf{U}_t$ as

$$\begin{aligned} \mathbf{F}_t &= f([\mathbf{o}_t^p, \mathbf{z}_t^h, \hat{\mathbf{v}}_t, \hat{\mathbf{p}}_t, \mathbf{L}_{t-1}]), \\ \mathbf{U}_t &= g([\mathbf{o}_t^p, \mathbf{z}_t^h, \hat{\mathbf{v}}_t, \hat{\mathbf{p}}_t, \mathbf{L}_{t-1}]), \end{aligned} \tag{4.1}$$

where both tensors match the shape of the window and are bounded to $(0, 1)$ by a sigmoid. Through $\mathbf{L}_{t-1}$ the networks read the window before updating it, so $\mathbf{F}_t$ acts as a forget

gate on the stored content and $\mathbf{U}_t$ writes new information into it. Following the update rule of the HMF, the window content is then overwritten with

$$\tilde{\mathbf{L}}_t = \mathbf{L}_{t-1} \odot \mathbf{F}_t + \mathbf{U}_t, \quad \mathbf{L}_t = \frac{\tilde{\mathbf{L}}_t}{\max(\tilde{\mathbf{L}}_t) + \epsilon}, \quad (4.2)$$

where $\odot$ denotes the element-wise product and the normalization with $\epsilon = 0.02$ keeps all entries in $[0, 1]$. All entries outside $\mathcal{W}_t$ remain unchanged.

Observations made in different parts of the maze therefore modify different parts in the memory, and a written cell keeps its latent exactly until the robot returns to the corresponding location, independent of how much time passes in between. The calibration through $\mathbf{F}_t$ lets the policy decide how much of the previously stored information to keep when it revisits a location, and the spatial layout of $\mathbf{M}_t$ allows a direct visual comparison with the true maze, which we use in chapter 6.

4.4. RL Training Setup in Simulation

We train policy and critic with PPO in 5 000 parallel MJX [40] environments with 300 000 pre-generated maze layouts. The full objective combines the clipped surrogate and value losses of PPO with the maze reconstruction and velocity estimation losses

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{\text{PPO}} + \lambda_{\text{maze}} \mathcal{L}_{\text{maze}} + \lambda_{\text{vel}} \mathcal{L}_{\text{vel}} \quad \lambda_{\text{maze}}, \lambda_{\text{vel}} \in \mathbb{R} \\ \mathcal{L}_{\text{maze}} &= -\frac{1}{n_w} \sum_{i=1}^{n_w} [y_{t,i} \log \hat{y}_{t,i} + (1 - y_{t,i}) \log(1 - \hat{y}_{t,i})], \\ \mathcal{L}_{\text{vel}} &= \|\mathbf{v}_t - \hat{\mathbf{v}}_t\|^2, \end{aligned} \quad (4.3)$$

where $\mathbf{y}_t \in \{0, 1\}^{n_w}$ indicates which of the n_w possible wall segments of the maze exist, $\hat{\mathbf{y}}_t$ contains the wall probabilities predicted by the auxiliary head, and $\mathbf{v}_t$ is the simulated linear velocity of the trunk. $\mathcal{L}_{\text{maze}}$ is the Binary Cross Entropy (BCE) error between the predicted and the true wall layout, averaged over all n_w wall segments, so every wall of the maze contributes equally to the error. Its gradients reach the write functions f and g through the memory encoder m and train the memory to store the maze layout. The exploration task demands long-horizon credit assignment, as reaching the last unexplored cells at the end of an episode depends on decisions made many seconds earlier. Therefore, we use a high discount factor of $\gamma = 0.995$. Table A.1 lists all learning hyperparameters.

4.5. Rewards

Each training episode is either a goal episode, in which the robot is commanded toward a neighboring maze cell, or an exploration episode, in which it receives no command and must cover as much of the maze as possible. The reward r_t consists of task rewards, which depend on the episode type, and shaping penalties for the gait and the navigation. In exploration episodes, the robot collects a reward for being in rarely visited cells, which decays with the time already spent in the current cell, and a reward for the overall maze coverage. In goal episodes, it is instead rewarded for moving toward the commanded cell and reaching it. In addition, the maze reconstruction reward is active in both episode types and rewards the agent for correctly predicting the maze layout, based on the BCE error between the predicted and true wall distribution. The shaping penalties punish stalling, walking close to walls, joint limits, torques, action rates, and asymmetric gaits. Table A.2 lists all terms and coefficients.

Each term r_i is scaled by its coefficient w_i from Table A.2, by the timestep Δt , and the gait shaping penalties are additionally scaled by the task curriculum coefficient λ introduced in section 4.6, so that they do not suppress learning early in training. The total reward is computed as

$$r_t = \max \left(\sum_{i \in \mathcal{T}} w_i r_{t,i} + \lambda \sum_{j \in \mathcal{S}} w_j r_{t,j}, 0 \right) + w_{\text{alive}}, \quad (4.4)$$

where $\mathcal{T}$ contains the task, navigation and auxiliary shaping terms and $\mathcal{S}$ the gait shaping terms of Table A.2. Furthermore, a selection of rewards during training time are visualized in Figure A.1 - Figure A.7. Note that the sum of all reward terms is clipped at zero before a constant alive bonus w_{alive} is added outside the clipping. This ensures that the accumulated penalties can never drive the total reward negative, which would otherwise make early termination attractive, while the alive bonus guarantees a strictly positive signal for staying alive at every timestep.

4.6. Curricula

Learning to walk and to explore at the same time is difficult, as the exploration rewards provide no useful signal while the robot cannot walk yet. Therefore, we bootstrap training with goal episodes, in which the command $\mathbf{c}_t$ contains the position of a reachable neighboring maze cell in the robot’s base frame. The least visited neighboring cell is selected to

already bias the policy toward exploration, and once it is reached, the next goal is sampled immediately. The last entry of $\mathbf{c}_t$ flags the episode type, and in exploration episodes the command is the constant vector $[0, 0, 1]$, so the policy must decide on its own where to move.

The difficulty of training is controlled by two performance-based curriculum coefficients [41, 42]. The task curriculum $\eta \in [0, 1]$ and the terrain curriculum $\mu \in [0, 1]$ are single scalar coefficients that scale all curriculum-dependent parts of the training setup. The task coefficient η controls the fraction of exploration episodes, ramps up the shaping penalties, so that they do not suppress the learning of exploration early in training and scales domain randomization ranges. The terrain coefficient μ controls the fraction of the maze’s walls that are present and the sampling ranges of the environment properties, so the policy first learns to walk and follow goals in open spaces before it faces more and more complicated mazes.

Both coefficients are discretized into 100 levels with a base step size of $\Delta = 1/100$ and are updated only at episode termination. Success and failure of an episode are determined by the fraction of visited cells $\rho \in [0, 1]$ and the episode length T relative to the maximum episode length $T_{\max}$,

$$\begin{aligned} S &= (\rho \geq \rho^+) \wedge (T > 0.9 T_{\max}), \\ F &= (\rho < \rho^-) \vee (T < 0.6 T_{\max}), \end{aligned} \quad (4.5)$$

with the thresholds $(\rho^+, \rho^-) = (0.8, 0.4)$ in goal episodes and $(0.6, 0.3)$ in exploration episodes. The task coefficient and its step multiplier $k \in \mathbb{N}$ are then updated as

$$\lambda \leftarrow \begin{cases} \text{clip}(\lambda + k \Delta, 0, 1) & \text{if } S, \\ \text{clip}(\lambda - \frac{1}{4} \Delta, 0, 1) & \text{if } F \wedge \lambda < 0.95, \\ \lambda & \text{otherwise,} \end{cases} \quad k \leftarrow \begin{cases} k + 1 & \text{if } \lambda \text{ increased,} \\ 1 & \text{if } \lambda \text{ decreased,} \\ k & \text{otherwise.} \end{cases} \quad (4.6)$$

Consecutive successes therefore grow the step size linearly, which lets the curriculum accelerate through difficulty levels the policy already masters and drastically reduces the time needed to converge, while a single failure resets the step size and only mildly lowers the difficulty, still forcing careful progression once the policy starts to fail. The terrain coefficient μ follows Equation 4.6 with its own step multiplier, with the additional condition for an increase that the terrain difficulty sampled in the terminated episode was at least 0.9μ , ensuring that the policy has actually demonstrated success near the current difficulty level before it is raised further.

At the end of training, effectively all episodes are exploration episodes, and the deployed policy never depends on an external goal.

4.7. Domain Randomization

To enable sim-to-real transfer to the real robot, we apply extensive domain randomization during training, with all parameters listed in Table A.3. We randomize the robot model, including the trunk mass, the center of mass, the feet sizes, the collision geometry as well as the joint damping, armature, and friction. For the environment, we randomize the gravity vector, the ground friction, and the contact compliance, and for the actuation, we randomize the motor strength and apply action delays. We further randomize the initial state of the robot by adding noise to its initial joint positions and velocities, its base orientation, and its start position. On the observation side, we add noise to all proprioceptive observations and distort the height samples with noise, tilts, and delays of up to 60 ms. The position estimate is perturbed by up to ± 0.20 m, corresponding to one memory cell, so that the policy learns to tolerate imperfect localization. Finally, we push the robot with velocities of up to 0.3 m/s to simulate external disturbances. Most randomization ranges grow with the curriculum coefficient η , so that the policy faces increasingly difficult dynamics as its performance improves, while the observation noise is applied at full strength throughout training, as detailed in the caption of Table A.3. For the policy deployed on the real robot, we further widen several randomization ranges and additionally randomize the controller gains and the contact time constant, denoted by the values in parentheses in Table A.3, to make the deployed policy even more robust against the unmodeled dynamics of the real world.

4.8. Maze Generation

To learn a policy that explores mazes without overfitting to certain patterns in the maze generation, we need a maze generation algorithm that is unbiased. Creating mazes is closely related to spanning trees, as a perfect maze, meaning a fully connected maze without loops, can be described as a spanning tree over the grid of maze cells. A uniform spanning tree is a spanning tree sampled with uniform probability from the set of all possible spanning trees. This property allows us to generate unbiased, pattern-free mazes in which each maze layout is equally probable. Wilson’s algorithm [43] generates such uniform spanning trees using loop-erased random walks. We base our maze generation on Wilson’s algorithm, but as loops are critical to our experiments, for example to test whether a policy is capable of breaking out of already visited loops and exploring uniformly, we modify the output of Wilson’s algorithm and reintroduce loops by opening dead ends

with a fixed probability. During this braiding step, we reject wall openings that would create 2×2 open areas, so that the maze structure of narrow corridors is preserved. Note that this post-processing step trades the strict uniformity of the underlying spanning tree for the desired loop structure, while the unbiasedness of the base layout is retained. Finally, we check each generated maze against all previously generated mazes and discard duplicates, so that the training set consists of unique layouts. Algorithm 1 summarizes the full procedure. The resulting 5×5 maze structure is then loaded into simulation at every episode start uniformly sampled from 300_000 pre-generated mazes.

Algorithm 1 Maze generation based on Wilson’s algorithm with braiding

Require: grid size $R \times C$, number of mazes M , braid probability p

Ensure: set $\mathcal{D}$ of M unique maze layouts

```

1:  $\mathcal{D} \leftarrow \emptyset$ 
2: while  $|\mathcal{D}| < M$  do
3:   // Wilson’s algorithm: uniform spanning tree  $T$ 
4:    $T \leftarrow \{v\}$  with  $v$  sampled uniformly from all cells
5:   while  $T$  does not contain all cells do
6:     sample start cell  $u \notin T$  uniformly
7:     perform a random walk from  $u$  until reaching  $T$ 
8:     erase all loops from the walk in the order they appear
9:     add the resulting loop-erased path to  $T$ 
10:  end while
11:  // Braiding: reintroduce loops
12:  for all dead-end cells  $c$  in  $T$  in random order do
13:    with probability  $1 - p$ : continue
14:     $W \leftarrow$  closed inner walls of  $c$  whose removal creates no  $2 \times 2$  open area
15:    if  $W \neq \emptyset$  then
16:      remove a wall sampled uniformly from  $W$ 
17:    end if
18:  end for
19:  // Uniqueness check
20:   $m \leftarrow$  wall tensor of the braided maze
21:  if  $m \notin \mathcal{D}$  then
22:     $\mathcal{D} \leftarrow \mathcal{D} \cup \{m\}$ 
23:  end if
24: end while
25: return  $\mathcal{D}$ 

```

5. Experimental Setup

In this chapter we introduce the setup of our experiments and the metrics used to evaluate our approach. Our experiments examine how LSMM explores compared to unstructured memory architectures, what the different memories actually retain about the environment, and whether the learned behavior transfers to the real robot. We compare LSMM against an LSTM, a Transformer, and an MLP with a fixed window of stacked past observations. For each architecture, we train three seeds on a single NVIDIA RTX 3090 GPU with 24 GB of VRAM for 2.5 billion environment steps, evaluate each policy on multiple simulation experiments, and deploy LSMM in a real-world setting. The resulting training times differ considerably between the architectures, as shown in Figure A.7: the LSTM-based policy is the most expensive to train at over 20 hours, while the fixed-window baseline is the fastest at slightly above 10 hours; our LSMM approach completes training in roughly 16 hours.

5.1. Baselines

Each baseline exchanges the memory module of our architecture with its own memory mechanism. Specifically, each baseline replaces the LSMM block in Figure 4.1 with its own component. All baselines receive the same observations as LSMM, including the position estimate, are trained with the same auxiliary losses and the hyperparameters of Table A.1, and therefore differ from LSMM only in their memory mechanism.

5.1.1. LSTM

The LSTM is the architecture closest to our approach, as it represents the standard recurrent architecture used in memory-enhanced RL. The idea behind this baseline is to directly compare our approach, which leverages structure inside the hidden state, against an approach without such structure. We use a hidden state vector of size 1000, as this is

the largest hidden state that fits onto our training setup. In comparison, our approach leverages a hidden state of size $50 \cdot 50 \cdot 3 = 7500$. The reason our approach fits this much larger hidden state into memory is that our gating functions are considerably small, as the gating matrices F_t and U_t only have a size of $5 \cdot 5 \cdot 3 = 75$ compared to 1000 in the LSTM.

5.1.2. FixedWindow

The most straightforward approach to build up memory is to stack the last observations together and feed them into the agent, where an MLP creates a feature vector out of the stacked history. This approach is limited by two factors. First, naive frame stacking can be very memory intensive, as the memory consumption grows strongly with longer windows. Second, the maximum observation history is limited by the unroll length of our training setup. Due to these memory restrictions and our unroll length of 20, we decided to use an observation history of the last 10 observations. Furthermore, we apply masking if an episode ends to restrict inter-episode information exchange.

5.1.3. Transformer

As transformers have a strong ability to select important information from a given sequence based on a query, they are also very interesting for memory-enhanced RL. Since the observation history is treated as a sequence, this approach is subject to similar restrictions as the FixedWindow approach. Our implementation follows the TrXL-I architecture [23] with a sequence length of the last 10 observations. Similar to FixedWindow, we apply masking on the sequence if an episode has terminated within these observations.

5.2. Simulation

We let each trained policy run in 100 unseen evaluation environments. Each environment consists of a unique maze constructed similarly to the mazes in the training setup but not used during training. Using these environments, we assess each policy on three properties, namely its exploration efficiency, its maze reconstruction capabilities, and its ability to encode environment properties that are randomized per maze cell.

5.2.1. Exploration Efficiency

First, we measure the exploration efficiency of each policy as the average amount of explored cells inside the maze. This metric indicates how well an agent is able to explore mazes. Second, we record the average visits per maze cell for each policy. As mazes are constructed randomly, this metric shows whether an agent merely learned a fixed pattern to walk through a maze or actually learns to adapt to the given maze. We therefore hypothesize that the more uniformly the average cell visits are distributed, the better the agent adapts to the maze and recognizes parts it has already explored. To quantify this, we leverage the KL divergence between the cell visit distribution p produced by each policy and a uniform distribution u , in which each cell is visited equally often.

$$\begin{aligned} D_{\text{KL}}(p \parallel u) &= \sum_{i=1}^n p_i \log \frac{p_i}{u_i} \\ &= \sum_{i=1}^n p_i \log \frac{p_i}{\frac{1}{n}} \\ &= \sum_{i=1}^n p_i \log (p_i \cdot n) \end{aligned} \tag{5.1}$$

Equation 5.1 shows the computation for this special case of the KL divergence. As the probability of each cell under the uniform distribution is $u_i = 1/n$, with n being the number of maze cells, the term can be simplified. Since p is normalized over the total number of visits, the absolute visit counts cancel out and only the shape of the distribution matters. A smaller KL divergence therefore states more uniform exploration, indicating that the agent adapts to the given maze instead of repeatedly revisiting the same parts.

5.2.2. Maze Reconstruction Capabilities

As described in section 4.2, we leverage an additional maze layout prediction head on the memory module to test the long-term memory capabilities of each architecture. In this evaluation, we store the output of this prediction head and compare it against the true maze layout. As the agent explores the maze throughout the episode, we compute the average reconstruction error at every timestep. A policy that is capable of keeping the layout inside its memory should therefore exhibit an error that shrinks over time, since the more time the agent spends in a maze, the better it should be able to predict the full layout.

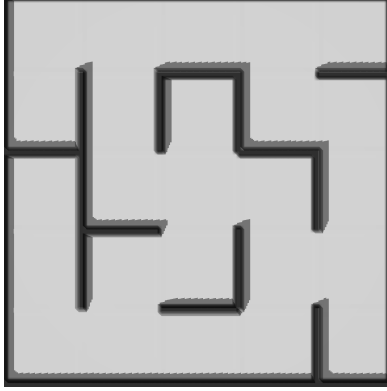


Figure 5.1.: Top-down view of the ground truth layout of the maze chosen to visually analyze the exploration and reconstruction capabilities of LSMM and the baselines. Black lines refer to walls inside the maze.

$$\mathcal{BCE}_{\text{maze}} = -\frac{1}{n_w} \sum_{i=1}^{n_w} [y_{t,i} \log \hat{y}_{t,i} + (1 - y_{t,i}) \log(1 - \hat{y}_{t,i})] \quad (5.2)$$

As our metric, we leverage the BCE error shown in Equation 5.2, which is similar to the definition of $\mathcal{L}_{\text{maze}}$ in Equation 4.3. Here, $\mathbf{y}_t$ refers to the true maze wall layout, $\hat{\mathbf{y}}_t$ to the predicted layout at each timestep, and n_w to the size of the maze wall layout. In addition, we visualize the prediction capabilities of each policy in a selected maze shown in Figure 5.1. We chose this maze because it combines multiple complex parts the agent has to conquer. The maze contains dead ends in the left part, in which the agent must learn to turn around in order to explore further. The maze additionally contains multiple loops of both shorter and longer length. An agent that cannot remember where it has already been over a long time horizon could easily get stuck in such loops. Furthermore, the maze is complex enough that randomly predicting a maze layout would not benefit the agent at any time.

5.2.3. Encoding Environment Properties

To test whether the spatial memory can also support predictions beyond the maze layout, we train an auxiliary head, similar to the maze prediction head, to estimate locally varying environment properties, including gravity, ground friction, roughness, and ground

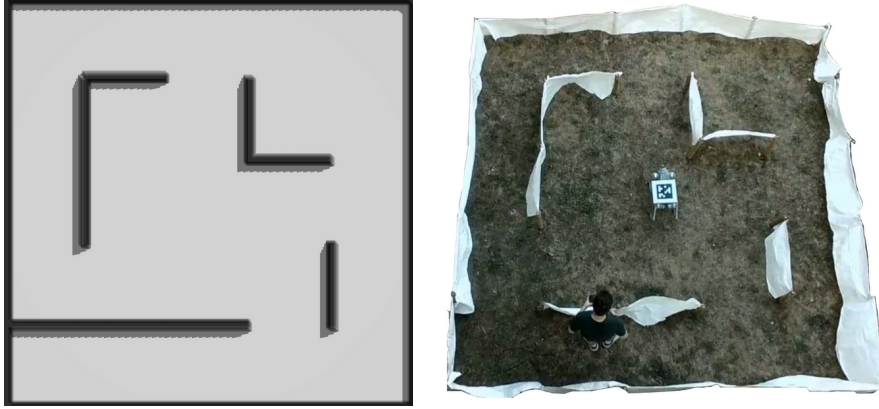


Figure 5.2.: Ground truth maze layout used in deployment (left) and the real-world maze (right). The ground truth layout contains an additional wall in the lower left part that is missing in the real-world maze. Since the artificial height samples are built from the ground truth layout, the agent observed this wall during the deployment run, and the left layout is therefore used for the metric computation.

damping. For each cell in a maze, we sample each property uniformly from a range scaled by the performance-based curriculum.

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{Z}_i - Z_i| \quad (5.3)$$

We compare the memory capability against random guessing of the environment properties and use the Mean Absolute Error (MAE) shown in Equation 5.3 as our comparison metric, where a lower MAE indicates a better prediction. Here, $\hat{Z}_i$ denotes the prediction vector of all environment properties and Z_i the true property vector. In our evaluation, we report the prediction capability per environment property.

5.3. Real World

We deploy the LSMM policy on the Unitree Go2 in an unseen outdoor maze, as shown in Figure 5.2. The goal of this experiment is to demonstrate that the exploration behavior and the maze layout prediction of LSMM carry over from simulation to a real robot. We

therefore deploy only LSMM and do not evaluate the baselines in the real world, as the comparison between the architectures is covered by the simulation experiments. Due to constraints on the available space, we reduce the size of the maze from 10×10 m to 5×5 m and therefore train a new policy that is capable of traversing the smaller maze. Compared to our simulation setup, several challenges arise in this environment. A policy conquering such a maze must be far more cautious about where to step to avoid crashing into walls. We therefore introduce additional penalties for being too close to a wall or deviating too much from the center line of the maze. Apart from the maze dimensions and these additional penalties, the architecture, observations, and training procedure remain unchanged. We get the position estimate from an AprilTag [44] on the robot and run the policy locally on the onboard CPU, see Figure 5.2 (right). Furthermore, we leverage the position estimate to build an artificial height map around the robot’s trunk, matching the height samples in simulation. Similar to simulation the policy starts in the maze center and does not receive a goal or an externally generated navigation command. It explores the maze fully autonomously. To test whether the deployed policy behaves similarly to LSMM in simulation, we record the average visits per cell and the maze layout reconstruction error and compare them to the simulation results.

6. Results

In this chapter, we present the results of evaluating our approach and the baselines in the experimental settings described in chapter 5. We start by comparing our approach to the baselines in simulation and then evaluate the applicability of our approach in the real world.

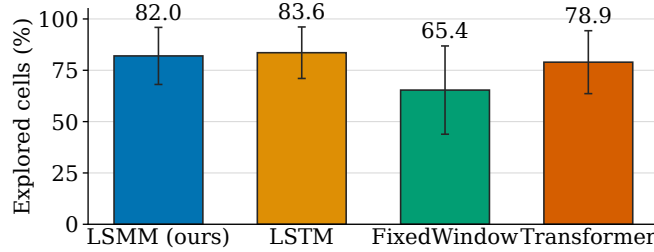


Figure 6.1.: Percentage of explored cells at the end of an episode. Bars show the mean over 100 unseen evaluation mazes and three seeds per architecture, whiskers the standard deviation. All architectures reach a comparable coverage.

6.1. Simulation

Figure 6.1 shows the fraction of explored cells at the end of an episode. LSMM, LSTM, and the Transformer reach a comparable coverage of 78.9 % to 83.6 %, while FixedWindow slightly falls behind with an average of 65.4 % of explored cells. We therefore derive that pure exploration performance in our mazes does not require a spatial memory.

However, the coverage alone does not reveal how this exploration is achieved. The cell visit distributions in Figure 6.2 are visualized as heat maps, where a darker cell color indicates more average visits to that cell. Furthermore, the KL divergence between each

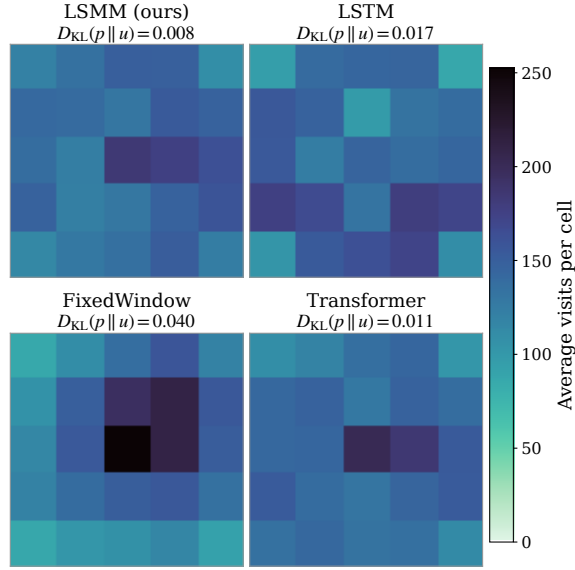


Figure 6.2.: Average number of visits per maze cell across all evaluation episodes. $D_{KL}(p \parallel u)$ measures the divergence between the normalized visit distribution p and a uniform distribution u , where lower values mean more even exploration. LSMM spreads its visits most evenly over mazes.

visit distribution p and the uniform distribution u is shown. Here, a first difference between the architectures is revealed. LSMM spreads its visits most evenly over the maze, with a KL divergence of 0.008 compared to 0.011 to 0.017 for the Transformer and LSTM, while FixedWindow concentrates its visits on the cells around the start. We can also observe a difference in the edge cells of the maze. Comparing LSMM and LSTM, the corner cells are visited slightly more often by the LSMM policies, which underlines the more uniform visit distribution as well. Following our hypothesis from chapter 5, this more uniform distribution indicates that LSMM adapts to the given maze and recognizes parts it has already explored instead of merely following a fixed walking pattern.

Coverage alone also does not show what a policy has remembered. We therefore examine the maze reconstruction error over the episode, shown in Figure 6.3. LSMM is the only architecture that significantly reduces the error while exploring, from 0.33 to 0.11, more than three times lower than any baseline. LSTM slightly reduces the error within the first ten seconds but stagnates quickly afterwards. The Transformer and FixedWindow show a similar behavior, as both approaches plateau at an error higher than initial guessing,

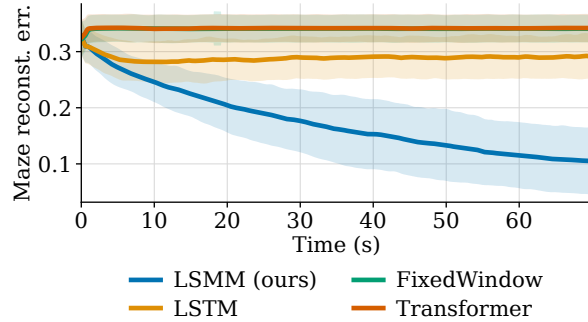


Figure 6.3.: Maze reconstruction error over the course of an episode. Lines show the mean over 100 unseen evaluation mazes and three seeds, shaded areas the standard deviation. LSMM is the only architecture that significantly reduces the error while exploring.

showing that an observation history of the last 10 observations is not enough to predict any meaningful maze layout.

Figure 6.4 visualizes the maze layout predictions of all approaches in the chosen maze stated in subsection 5.2.2 at three timesteps, $t = 0.2\text{ s}$, $t = 35.0\text{ s}$, and $t = 70.0\text{ s}$. Therefore the predictions are shown at the beginning, during, and at the end of the episode. Furthermore, the trajectory the agent took up to each timestep is visualized as well. The maze predicted by LSMM fills in wall by wall as the robot walks, until it closely matches the ground truth at the end of the episode. The baselines, in contrast, either never predict more than a few walls near their current position or hallucinate walls in free spaces. These qualitative observations match the quantitative reconstruction errors discussed above. When visually inspecting the behavior of the policies, the baselines explore more reactively and forget the areas they have seen before, which can even lead to circling around walls in loops. LSMM avoids returning to previously explored areas by building a persistent map of the maze. It rarely hallucinates walls and keeps true walls in memory as long as it has explored the corresponding cell enough. Across all architectures, the policies act mostly restless and have a tendency to pace from cell to cell at $0.8\text{--}1.2\text{ m/s}$ to maximize the intrinsic visitation reward.

To test possible extensions of our LSMM approach, we train a policy with a second auxiliary head on the memory module that predicts the environment properties described in chapter 5. Figure 6.5 shows the prediction capability of our memory compared to randomly guessing the properties. The environment head trained on our memory module

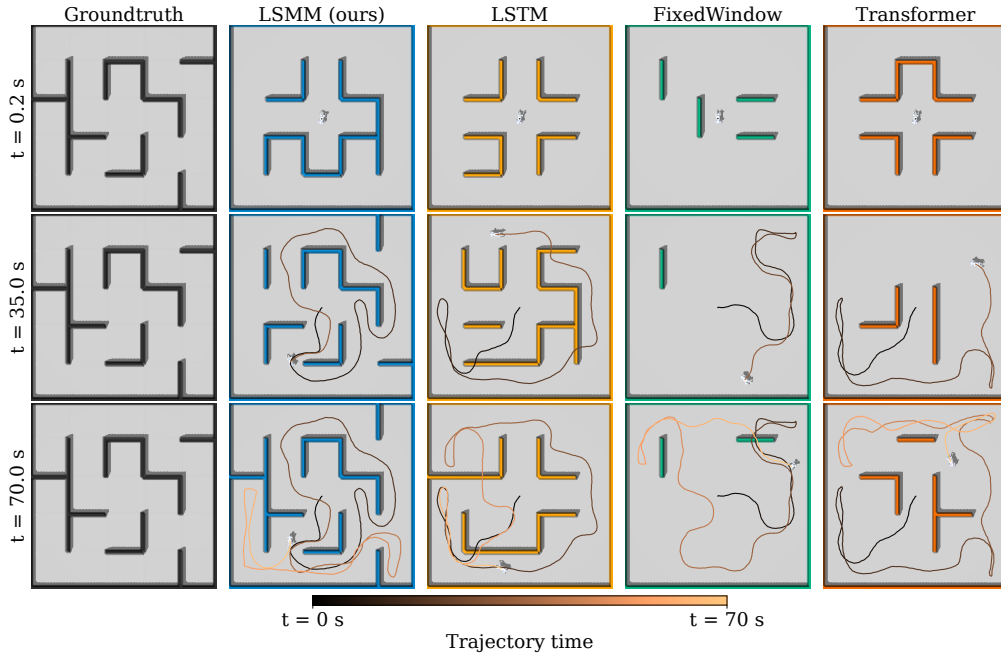


Figure 6.4.: Predicted maze layout at three times during an evaluation episode. The trajectory walked up to each time is overlaid, colored from black to orange by trajectory time. The prediction of LSMM fills in wall by wall as the robot explores and closely matches the ground truth at the end of the episode, while the baselines struggle to remember the walls in the areas they have been before.

shows a consistently lower error than random guessing across the evaluated properties. From this result, we reason that the policy can not only store where it has been but also aspects of the world it experienced there. We suspect that a larger latent size for the memory cells would allow better estimations and even more predicted properties. Storing such environment properties can be interesting in scenarios in which the policy has to adapt its gait for better traversability, or for route planning to decide where to walk next. However, these scenarios are not evaluated in this work and remain promising directions for future work.

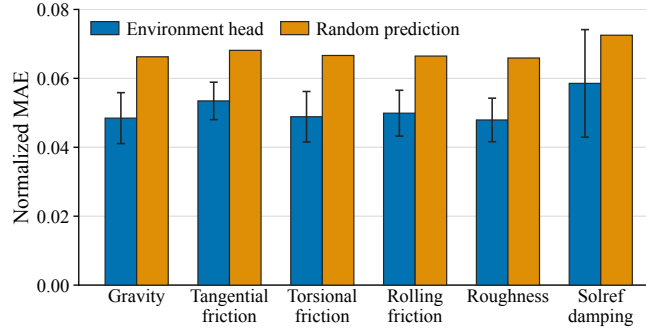


Figure 6.5.: Auxiliary prediction of spatially varying environment properties. Bars show the normalized mean absolute error over all visited cells at the final time step.

6.2. Real World

For the real-world deployment, the policy is deployed in the 5×5 m outdoor maze. The robot explores the maze fully autonomously, as it selects directions at intersections, traverses longer corridors, and continues exploring after returning to previously visited parts of the maze. The snapshot of the matrix memory in Figure 1.1 shows how the policy modifies the cells along its trajectory, while cells outside the visited region retain their initial value. The same structure is visible in the time spent in each cell shown in Figure 6.6, where the regions that received no matrix memory updates are also the ones the robot never entered. Compared to the simulation results, the deployment run distributes its visits less uniformly and follows a more repetitive pattern. However, the visit distribution covers enough of the maze to reduce the reconstruction error similarly to the simulation results, as shown in Figure 6.7. As the maze is significantly smaller than in the simulation setup, the policy observes a higher fraction of it early on, which leads to a faster error reduction. During the deployment runs, we noticed that the policy sometimes tries to cut corners or even touch the walls, which we could only reduce with strong penalties on cell centering and wall proximity. This is necessary in our more fragile real-world maze setup, but with proper solid walls, it might be beneficial to let the policy touch its surroundings as another form of re-orientation and grounding of its predictions, as long as the impact forces are mild.

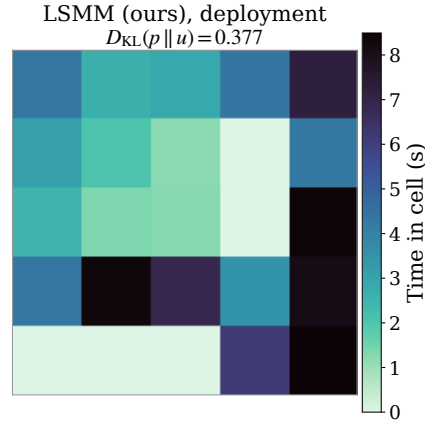


Figure 6.6.: Time spent in each cell by our deployed LSMM policy. Compared to the simulation results, this deployment run distributes its visits less uniformly and follows a more repetitive pattern, which is also visible in the matrix memory in Figure 1.1 (b).

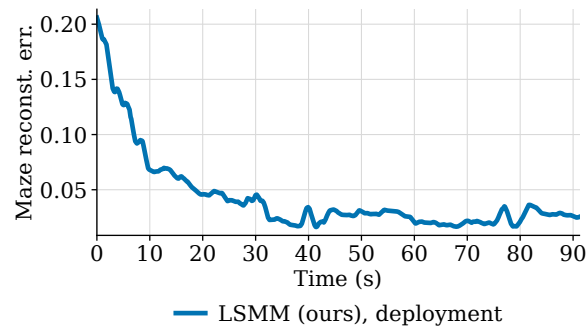


Figure 6.7.: Maze reconstruction error over deployment time of our LSMM policy. The line shows the reconstruction error of the deployment run shown in Figure 1.1. The policy reduces the error significantly within the first 30 s.

7. Discussion

Our results show that structuring the hidden state spatially changes what a policy remembers rather than how much it explores. All architectures reach comparable maze coverage, but only LSMM retains and reinforces the maze layout over time, which confirms that pure exploration in our mazes does not require spatial memory while persistent knowledge of the environment does. A further benefit of the architecture lies in its efficiency. Since the HMF update only modifies a local window per timestep, the gating functions remain small, which makes it possible to fit an overall larger hidden state into memory. This is especially beneficial for setups leveraging heavily parallelized simulation, which makes learning such policies feasible in the first place.

However, our approach has limitations. In the provided environment, the LSMM memory covers a fixed area that is aligned with the environment frame and sized to the maze, so exploring environments of unknown size would require growing or shifting the memory with the robot. The update further depends on an external position estimate, and although training with ± 0.20 m of noise makes the policy tolerate imperfect localization, we do not yet estimate the position from onboard sensing alone. Finally, our mazes are flat and small enough that reactive strategies reach similar coverage, so we expect the benefit of a spatial memory for pure exploration to grow with the size of the environment.

8. Conclusion & Outlook

8.1. Conclusion

We introduced LSMM, a spatially indexed latent memory that lets a single policy explore unseen mazes end-to-end, from onboard observations to joint-level actions. By applying the HMF update only to a local window centered on the robot’s position estimate, information about a location persists in the memory until the robot returns there. In our experiments, unstructured recurrent and finite-history baselines match LSMM in pure maze coverage, but they explore reactively and forget the layout they have seen. LSMM instead builds a persistent map of the maze while exploring, reduces the reconstruction error by a factor of three, spreads its visits most evenly over the maze, and additionally stores physical properties of the environment. We deployed the policy on a Unitree Go2 and demonstrated goal-free exploration of a real outdoor maze without an explicit map, planner, or waypoint commands.

8.2. Outlook

We see three directions to build upon the current results of our approach.

First, the approach can be extended to general unseen environments of the kind an agent would encounter in real-world settings. Scaling to larger mazes is an intuitive first step, as the benefit of a spatial memory grows with the size of the environment. The local updates could help to build a robust and persistent map directly inside the hidden state even as the environment grows. Beyond mazes, the same mechanism could be applied to more natural terrain, where the agent must identify which regions are traversable and retain this information in its memory, for instance steep, rough, or slippery areas it has already encountered.

Building on such a map, we also want to test whether an agent can learn to leverage its memory for efficient path planning. A particularly interesting scenario is a two-fold task in which the agent first explores an environment, filling its matrix memory as completely as possible, and in a second stage uses the written memory to autonomously find a fast path from its current location to a previously seen one. Encoding terrain properties into the memory would further enrich this setting: the agent could then decide where to walk and, for example, prefer a longer but safer route over a shorter one that requires climbing or crossing slippery ground.

Finally, predicting the current position from the memory itself is a very interesting direction, in which the estimate would also be used to write into the memory at the predicted location. We touched on this idea throughout this thesis, but major challenges arise from combining prediction and writing in this way. The problem can be described as a chicken-and-egg problem, as the agent needs a good memory to estimate correct position, but without a good position estimate, building up a reliable memory is hard. Furthermore, off-policyness arises in the training setup, as an improving localization prediction changes the memory content, leading to stale updates during the loss computation.

To summarize, these are possibilities and challenges that can be tackled to build memory-enhanced policies for quadrupedal locomotion and navigation that autonomously walk and explore an environment, leveraging the benefit of end-to-end optimization to find smarter and more energy-efficient ways to move through the world.

Bibliography

- [1] Z. Zhuang, Z. Fu, J. Wang, C. Atkeson, S. Schwertfeger, C. Finn, and H. Zhao, “Robot parkour learning,” in *Conference on Robot Learning (CoRL)*, 2023.
- [2] X. Cheng, K. Shi, A. Agarwal, and D. Pathak, “Extreme parkour with legged robots,” in *RoboLetics: Workshop on Robot Learning in Athletics@ CoRL 2023*, 2023.
- [3] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, “Anymal parkour: Learning agile navigation for quadrupedal robots,” *Science Robotics*, vol. 9, no. 88, p. eadi7566, 2024.
- [4] M. Stasica, A. Bick, N. Bohlinger, O. Mohseni, M. J. A. Fritzsche, C. Hübler, J. Peters, and A. Seyfarth, “Bridge the gap: Enhancing quadruped locomotion with vertical ground perturbations,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13370–13370, IEEE, 2025.
- [5] N. Bohlinger, J. Kinzel, D. Palenicek, L. Antczak, and J. Peters, “Gait in eight: Efficient on-robot learning for omnidirectional quadruped locomotion,” *arXiv preprint arXiv:2503.08375*, 2025.
- [6] N. Bohlinger, G. Czechmanowski, M. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo, “One policy to run them all: an end-to-end learning approach to multi-embodiment locomotion,” *Conference on Robot Learning*, 2024.
- [7] N. Rudin, D. Hoeller, M. Bjelonic, and M. Hutter, “Advanced skills by learning locomotion and local navigation end-to-end,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2497–2503, IEEE, 2022.
- [8] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA’97. Towards New Computational Principles for Robotics and Automation*, pp. 146–151, IEEE, 1997.

-
-
- [9] P. Mirowski, R. Pascanu, F. Viola, H. Soyer, A. J. Ballard, A. Banino, M. Denil, R. Goroshin, L. Sifre, K. Kavukcuoglu, *et al.*, “Learning to navigate in complex environments,” *arXiv preprint arXiv:1611.03673*, 2016.
- [10] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [11] K. Cho, B. Van Merriënboer, D. Bahdanau, and Y. Bengio, “On the properties of neural machine translation: Encoder–decoder approaches,” in *Proceedings of SSST-8, eighth workshop on syntax, semantics and structure in statistical translation*, pp. 103–111, 2014.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [13] G. Ji, J. Mun, H. Kim, and J. Hwangbo, “Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4630–4637, 2022.
- [14] D. Li, B. Ai, N. Bohlinger, J. Peters, H. I. Christensen, and H. Su, “Online embodiment adaptation for quadrupedal locomotion,” 2026.
- [15] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, “Elevation mapping for locomotion and navigation using gpu,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2273–2280, IEEE, 2022.
- [16] D. Aditya, J. Huang, N. Bohlinger, P. Kicki, K. Walas, J. Peters, M. Luperto, and D. Tateo, “Robust localization, mapping, and navigation for quadruped robots,” in *2025 European Conference on Mobile Robots (ECMR)*, pp. 1–8, IEEE, 2025.
- [17] M. Mattamala, J. Frey, P. Libera, N. Chebrolu, G. Martius, C. Cadena, M. Hutter, and M. Fallon, “Wild visual navigation: Fast traversability learning via pre-trained models and online self-supervision,” *Autonomous Robots*, vol. 49, no. 3, p. 19, 2025.
- [18] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, “Backpropagation applied to handwritten zip code recognition,” *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.

-
-
- [19] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems* (S. Solla, T. Leen, and K. Müller, eds.), vol. 12, MIT Press, 1999.
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [21] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” 2022.
- [22] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” 2018.
- [23] E. Parisotto, H. F. Song, J. W. Rae, R. Pascanu, C. Gulcehre, S. M. Jayakumar, M. Jaderberg, R. L. Kaufman, A. Clark, S. Noury, M. M. Botvinick, N. Heess, and R. Hadsell, “Stabilizing transformers for reinforcement learning,” 2019.
- [24] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, “On the properties of neural machine translation: Encoder-decoder approaches,” *CoRR*, vol. abs/1409.1259, 2014.
- [25] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.
- [26] H. Le, D. Nguyen, K. Do, S. Gupta, and S. Venkatesh, “Stable hadamard memory: Revitalizing memory-augmented agents for reinforcement learning,” in *International Conference on Learning Representations*, vol. 2025, pp. 73455–73472, 2025.
- [27] F. Al-Hafez, G. Zhao, J. Peters, and D. Tateo, “Time-efficient reinforcement learning with stochastic stateful policies,” in *International Conference on Learning Representations*, vol. 2024, pp. 49532–49568, 2024.
- [28] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, “Decision transformer: Reinforcement learning via sequence modeling,” 2021.
- [29] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” 2024.

-
-
- [30] S. Huang, J. Hu, Z. Yang, L. Yang, T. Luo, H. Chen, L. Sun, and B. Yang, “Decision mamba: Reinforcement learning via hybrid selective sequence modeling,” 2024.
- [31] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter, “xlstm: Extended long short-term memory,” 2024.
- [32] “MuJoCo XLA (MJX) - MuJoCo Documentation — mujoco.readthedocs.io.” <https://mujoco.readthedocs.io/en/stable/mjx.html>. [Accessed 04-08-2025].
- [33] NVIDIA, “Isaac Sim.”
- [34] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.
- [35] X. Cheng, K. Shi, A. Agarwal, and D. Pathak, “Extreme parkour with legged robots,” 2023.
- [36] H. Kim, H. Oh, J. Park, Y. Kim, D. Youm, M. Jung, M. Lee, and J. Hwangbo, “High-speed control and navigation for quadrupedal robots on complex and discrete terrain,” 2025.
- [37] C. Zhang, J. Jin, J. Frey, N. Rudin, M. Mattamala, C. Cadena, and M. Hutter, “Resilient legged local navigation: Learning to traverse with compromised perception end-to-end,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 34–41, IEEE, 2024.
- [38] D. S. Chaplot, D. Gandhi, S. Gupta, A. Gupta, and R. Salakhutdinov, “Learning to explore using active neural slam,” *arXiv preprint arXiv:2004.05155*, 2020.
- [39] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, “Asymmetric actor critic for image-based robot learning,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2018.
- [40] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.
- [41] B. Ai, L. Dai, N. Bohlinger, D. Li, T. Mu, Z. Wu, K. Fay, H. I. Christensen, J. Peters, and H. Su, “Towards embodiment scaling laws in robot locomotion,” *Conference on Robot Learning (CoRL)*, 2025.
- [42] N. Bohlinger and J. Peters, “Multi-embodiment locomotion at scale with extreme embodiment randomization,” *arXiv preprint arXiv:2509.02815*, 2025.

-
-
- [43] D. B. Wilson, “Generating random spanning trees more quickly than the cover time,” in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC ’96, (New York, NY, USA), p. 296–303, Association for Computing Machinery, 1996.
- [44] E. Olson, “AprilTag: A robust and flexible visual fiducial system,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3400–3407, IEEE, May 2011.

A. Appendix

To ensure a fair comparison, we train all memory architectures in our experiments with the same learning hyperparameters, which are listed in Table A.1.

Table A.2 lists all reward terms and their coefficients. All terms are scaled with their coefficients and summed up, and the result is clipped to be above zero before the constant alive bonus is added outside the clipping.

Table A.3 lists all domain randomization and observation noise parameters, with units provided where applicable. Deviations for the deployment policy are stated in parentheses.

Figure A.1 - Figure A.7 show curriculum convergence, rewards during training and training time of the LSMM approach aswell as baselines used for comparison.

Table A.1.: Hyperparameters used in all experiments.

Hyperparameter	Value
Training steps	$2.5 \cdot 10^9$
Parallel environments	5 000
Steps per environment	20
Batch size	100 000
Minibatch size	25 000
Updates per batch	5
Discount factor γ	0.995
GAE λ	0.9
Clipping range	0.2
Entropy coefficient (linearly annealed)	$4 \cdot 10^{-4}$ to $1 \cdot 10^{-4}$
Actor optimizer	Adam
Critic optimizer	AdamW
Velocity estimator and wall predictor optimizer	AdamW
Actor learning rate	$3 \cdot 10^{-4}$
Critic learning rate	$5 \cdot 10^{-4}$
Velocity estimator and wall predictor learning rate	$3 \cdot 10^{-4}$
Gradient clip norm	1.0
Weight decay (AdamW)	$1 \cdot 10^{-4}$
λ_{maze}	0.5
λ_{vel}	0.1

Table A.2.: Reward terms and coefficients, used in Equation 4.4. The set $\mathcal{T}$ contains the task and navigation shaping terms, and the set $\mathcal{S}$ contains the gait shaping terms, which are scaled by the curriculum coefficient λ . $[x]_+ = \max(x, 0)$ and $\bar{\cdot}$ denotes the average over joints. n_{cell} counts the steps spent in the current cell, δ_i is the deviation from the cell center toward walled side i , $n_{\text{pairs}}^{\text{air}}$ counts leg pairs with both feet in the air, and t^{air} is the air time of the feet.

Reward	Term	Coefficient
<i>$\mathcal{T}$ — Task, goal episodes</i>		
Goal direction	$\frac{1}{2}(1 + \cos(\mathbf{v}_{xy}, \mathbf{g}_t)) \min(\ \mathbf{v}_{xy}\ , 1)$	0.04
Goal reached	$\mathbb{1}[d_{\text{goal}} < 0.25]$	0.0625
<i>$\mathcal{T}$ — Auxiliary reward</i>		
Maze reconstruction	$[0.3 - \ell_{\text{BCE}}]_+ / 0.3$	0.06
<i>$\mathcal{T}$ — Navigation shaping</i>		
Stalling	$-[0.1 - \ \mathbf{v}_{xy}\]_+$	0.1
Wall proximity	$-\mathbb{1}[d_{\text{wall}} < 0.35] e^{-5(d_{\text{wall}} - 0.15)}$	0.015
Cell centering	$-\sum_i [\delta_i]_+^2$	0.04
<i>$\mathcal{S}$ — Gait shaping</i>		
Vertical velocity	$-v_z^2$	0.04
Roll-pitch velocity	$-\ \boldsymbol{\omega}_{xy}\ ^2$	$1 \cdot 10^{-3}$
Roll-pitch position	$-\ \boldsymbol{\phi}_{xy}\ ^2$	0.2
Joint position limits	$-\overline{[\mathbf{q} - \mathbf{q}^{\text{max}}]_+ + [\mathbf{q}^{\text{min}} - \mathbf{q}]_+}$	0.8
Joint velocity limits	$-\overline{[\dot{\mathbf{q}}] - 0.9 \dot{\mathbf{q}}^{\text{max}}}_+$	0.1
Joint velocity	$-\overline{\dot{\mathbf{q}}^2}$	$8 \cdot 10^{-6}$
Joint acceleration	$-\overline{\ddot{\mathbf{q}}^2}$	$1 \cdot 10^{-7}$
Joint torque	$-\overline{(\boldsymbol{\tau} / \boldsymbol{\tau}^{\text{max}})^2}$	$6 \cdot 10^{-4}$
Power draw	$-\overline{[\boldsymbol{\tau} \dot{\mathbf{q}}]_+ / (\boldsymbol{\tau}^{\text{max}} \dot{\mathbf{q}}^{\text{max}})}$	$6 \cdot 10^{-4}$
Action rate	$-\overline{(\mathbf{a}_t - \mathbf{a}_{t-1})^2}$	0.06
Nominal position	$-\overline{(\mathbf{q} - \mathbf{q}^{\text{nom}})^2}$	0.2
Base height	$-\min((h - h^{\text{nom}})^2, 1)$	0.6
Air time	$-\overline{[0.3 - t^{\text{air}}]_+}$	0.06
Feet pair air	$-n_{\text{pairs}}^{\text{air}}$	0.01
Ground collision	$-1[\text{body below ground}]$	2.0
<i>Outside clipping</i>		
Alive	1	$w_{\text{alive}} = 0.03$

Table A.3.: DR and observation noise parameters grouped by category. DR parameters with a range are sampled uniformly, scaled by the curriculum coefficient η from the center of the range at level 0 to the full range at level 1. DR parameters marked with a factor f scale their nominal model value uniformly within $[1 - \eta f, 1 + \eta f]$. Observation noise values x are sampled uniformly from $[-x, x]$ and applied fully without curriculum scaling. Values in parentheses denote the deployment policy where it differs from the simulation policy.

Parameter	Range / Factor	Unit	Parameter	Range / Factor	Unit
<i>General</i>			<i>Model randomization</i>		
Randomization probability per step	0.001	–	Gravity	–8.81 – –10.81	m/s ²
<i>Action delay</i>			Additional trunk mass	–2.0 – 2.0	kg
Maximum delay	1 (3) steps	–	COM displacement	–0.01 – 0.01	m
Delay probability	0.05	–	Foot size	0.018 – 0.026	m
<i>Control randomization</i>			Joint damping	0.0 – 0.1 (0.0 – 0.15)	–
Motor strength	0.8 – 1.2 (0.7 – 1.3)	–	Joint armature	0.005 – 0.015	–
P-gain factor	1.0 (0.7 – 1.3)	–	Joint friction loss	0.0 – 0.4	–
D-gain factor	1.0 (0.7 – 1.3)	–	Tangential friction	$f = 0.5$ (0.6)	–
Asymmetric factor	1.0 (0.98 – 1.02)	–	Torsional friction	$f = 0.5$ (0.6)	–
Joint position offset	0.0 (–0.02 – 0.02)	rad	Rolling friction	$f = 0.5$ (0.6)	–
<i>Perturbation randomization</i>			Terrain damping	$f = 0.6$	–
Push velocity per axis	–0.3 – 0.3	m/s	Terrain stiffness	$f = 0.5$	–
Additive push probability	0.5	–	Contact time constant	– (1.8, log range)	–
Additive multiplier	1.5	–	Lateral gravity components	0.5	m/s ²
<i>Initial state randomization</i>			Vertical gravity component	0.1	m/s ²
Joint position noise	0.1 (0.2)	rad	Trunk geometry size	0.05 – 0.1	m
Joint position offset	0.01	rad	Trunk geometry length	0.05 – 0.15	m
Joint velocity scale	0.1 (0.25)	–	Lidar geometry size	$f = 0.5$	–
Linear velocity noise	0.1 (0.25)	m/s	<i>Height map observation noise</i>		
Angular velocity noise	0.1 (0.25)	rad/s	Z offset noise	0.02	m
Roll angle noise	0.05 (0.15)	rad	Tilt roll noise	0.2	rad
Pitch angle noise	0.05 (0.15)	rad	Tilt pitch noise	0.2	rad
Yaw angle noise	0.0 (0.125)	rad	Gaussian Z noise std	0.02	m
Start position noise	0.0 (0.2)	m	XY grid noise	0.02 (0.2)	m
			Maximum delay	3 steps	–
			Delay probability	0.05 (0.1)	–
			<i>Proprioceptive observation noise</i>		
			Joint position noise	0.01	rad
			Joint velocity noise	0.5	rad/s
			Trunk angular velocity noise	0.2	rad/s
			Gravity vector noise	0.05	rad
			Position estimate noise	0.2	m

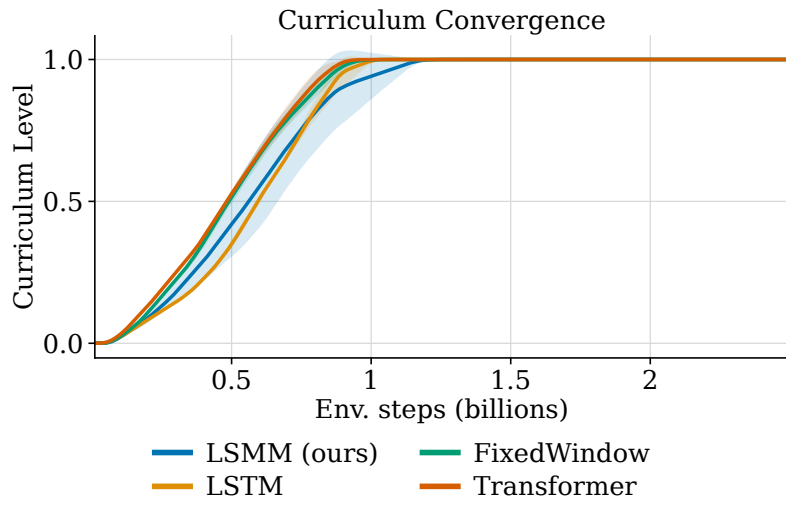


Figure A.1.: Curriculum convergence of all approaches during training, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation.

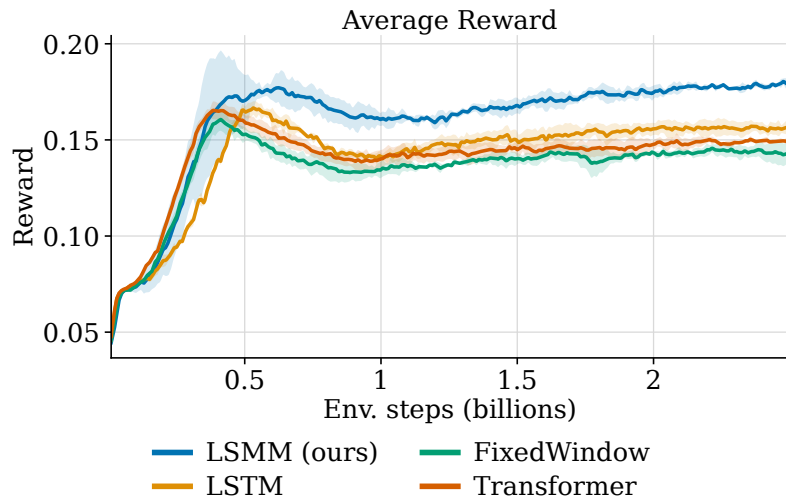


Figure A.2.: Average reward of all approaches during training, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation.

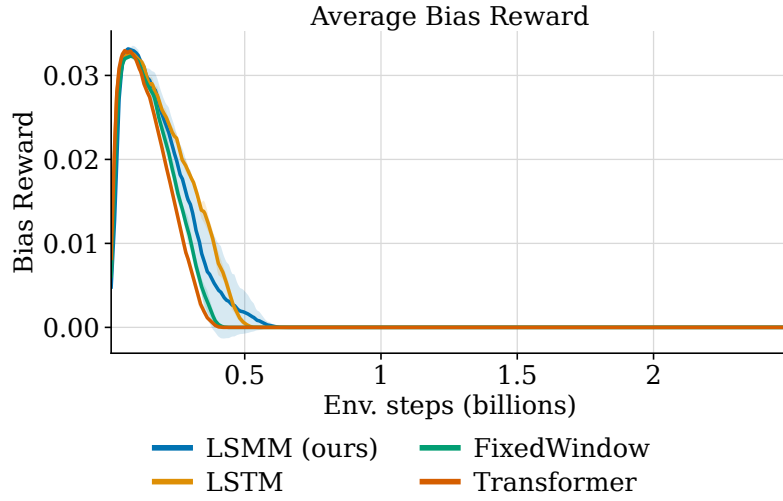


Figure A.3.: Average bias reward of all approaches during training, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation. The bias reward is the sum of the goal direction and goal reached rewards listed in Table A.2.

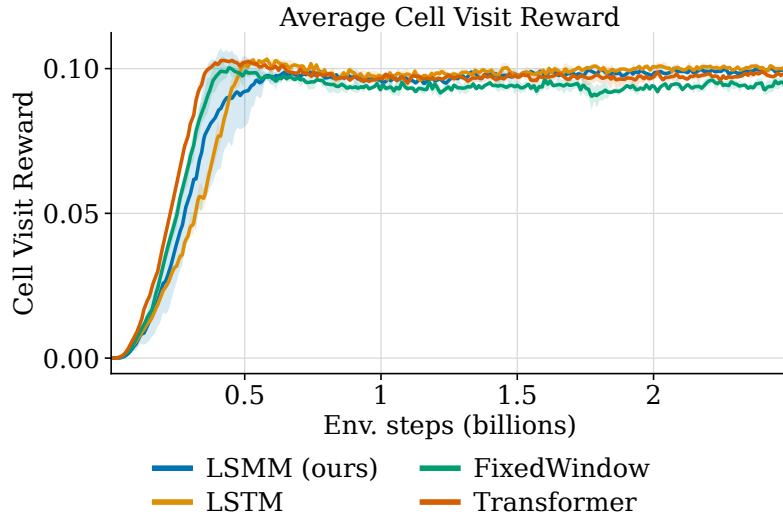


Figure A.4.: Average cell visit reward of all approaches during training, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation.

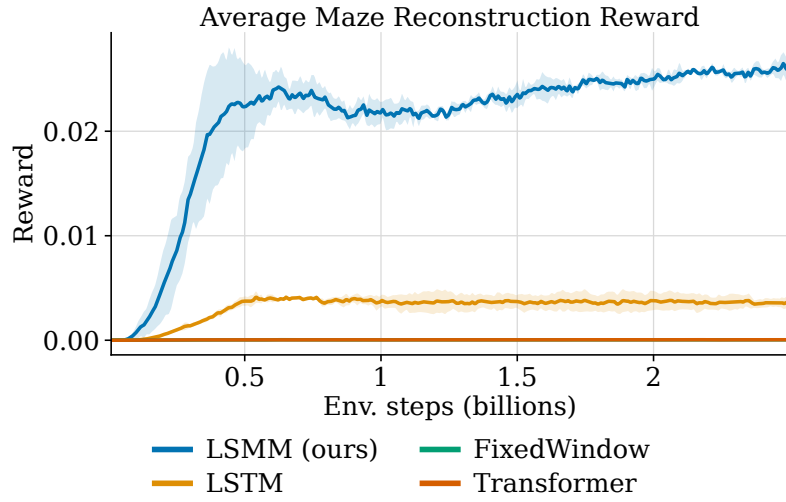


Figure A.5.: Average maze reconstruction reward of all approaches during training, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation. The FixedWindow and Transformer curves overlap at zero.

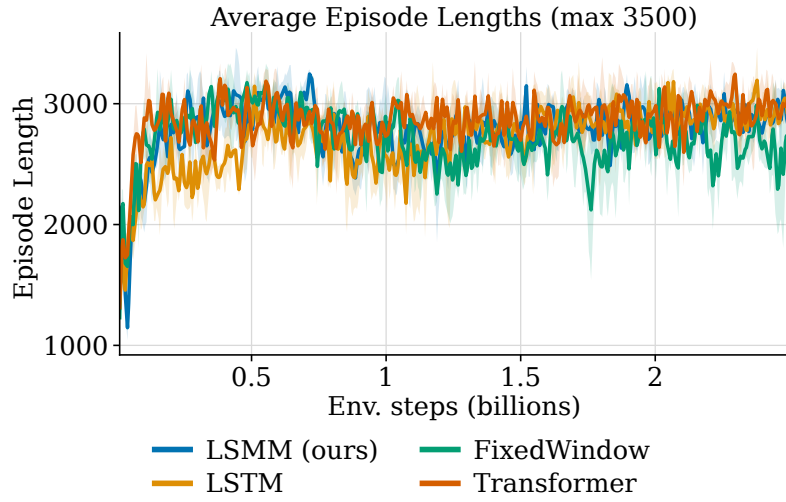


Figure A.6.: Average episode length of all approaches during training, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation.

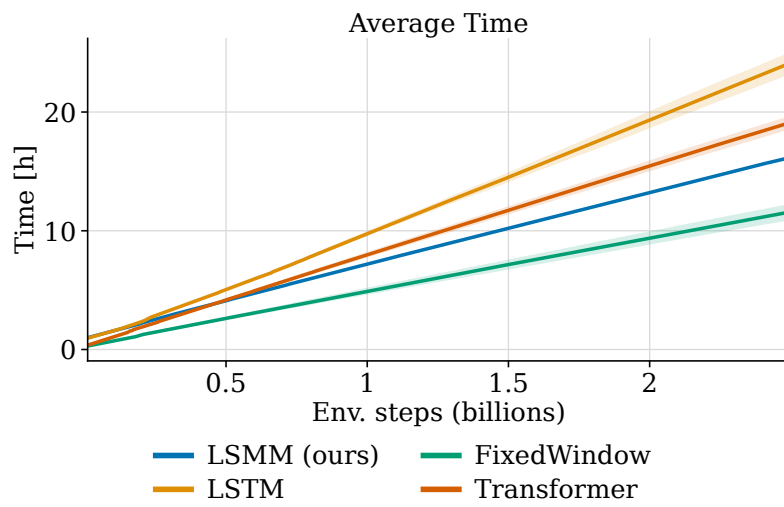


Figure A.7.: Cumulative wall-clock training time of all approaches in hours, plotted against environment steps. Lines show the mean over three seeds per approach; shaded areas indicate the standard deviation.